A Policy-Bounded Agentic Framework for Autonomous Governance of Cloud Data Pipelines

B Sai Krishna, Vignes VM, I Mahalakshmi, and Sri Harini MP

Abstract—A recent proposal governs cloud data pipelines with four policy-gated LLM agents and reports substantial gains over static orchestration, without saying which model-in-the-loop condition produced them. We implement the system and evaluate it with a real, remote language model in a 560-run campaign that crosses four injected fault scenarios with eight configurations: three non-agent comparators, the full system, and four single-agent ablations. With a live model, the full system changes mean time to recovery (MTTR) by −38% and cost by −56% relative to static orchestration. An assumed provisioning gap accounts for most of the cost change. Most of the MTTR change traces to a flaw we found afterwards in the simulated on-call engineer: it drew one delay per escalation and kept the fastest. The full system also increases manual interventions by 500%. Two idealized non-LLM baselines with fixed modelled latencies recover faster still, changing MTTR by −92% and −86%. When the same full configuration runs under a deterministic mock instead of the live model, recovery time falls to 0.193 s, about three orders of magnitude below the live figure, and the correct-mitigation score rises from 0.42 to 1.00. A mock evaluation and a live evaluation of the same system therefore support different conclusions. Graded against an oracle independent of the policy code, a generated adversarial corpus of 5547 cases yields containment ≥ 0.9990, but only after two contract-layer gaps it exposed were fixed. The evidence is bounded by a single host, synthetic data with injected faults, a simulated on-call engineer, one live model family, and baselines that are modelled rather than deployed.

Index Terms—LLM agents, cloud data pipeline governance, policy-as-code, Open Policy Agent, AIOps, evaluation methodology, adversarial evaluation, mock-vs-live evaluation validity

I. INTRODUCTION

Most pipeline failures are mundane. A column changes type upstream, a source goes quiet, a co-tenant takes the compute the pipeline needs, or a burst of ingress exceeds capacity. Foidl et al. counted 41 separate factors behind quality and reliability failures in real pipelines [1]. The usual fix is still a person on call, and much of the recovery time is simply the time that person needs to respond. Large language model (LLM) agents could cut this loop short. Several agent systems for cloud incidents already run in production [2], [3], [4], although most of them only recommend a diagnosis. When an agent is also allowed to act on production infrastructure, someone has to decide what it may and may not do.

One recent answer [5] uses four bounded agents for monitoring, optimization, schema management, and recovery. Each agent emits only a structured action proposal, and a policy engine must approve the proposal before anything executes. Against static orchestration, the authors report reductions of up to 45% in recovery time, about 25% in cost, and more than 70% in manual interventions. Four gaps separate these claims from the evidence offered for them.

1) Evaluation path. The study names several model families as backends but gives too little implementation detail to identify the model-in-the-loop condition behind its headline figures. This matters because work on the sim-to-real gap finds that simulated evaluation environments can inflate agent performance [6], [7], [8].
2) Baselines. The only comparator is static orchestration, with fixed allocations, predefined retries, human-initiated remediation, and autoscaling disabled. Mature non-LLM automation, however, already runs at production scale [9].
3) Adversarial testing. The safety case rests on the policy gate, but a stated bound is not a tested one. Agent-security benchmarks report high attack success against weak defenses [10], [11], and the policy language itself has been criticized as too weak for agentic governance [12].
4) Statistics and generality. No confidence intervals, effect sizes, or run counts are given. The paper treats GPT, Claude, and Gemini as interchangeable backends that differ only in verbosity and latency, but it shows no cross-model results. Other studies report that agent behavior does change with the model and the provider [13], [14].

We implement the system and ask five research questions.

RQ1: With a real model in the loop, does the full agent system outperform static orchestration on recovery time, cost, and manual interventions?
RQ2: Does it outperform cheap non-LLM automation, or only a slow human?
RQ3: Does it act correctly, and not merely quickly?
RQ4: Do the answers to RQ1–RQ3 change when the model is replaced by a deterministic mock?
RQ5: Does the policy gate contain adversarial proposals when judged against an oracle that is independent of the policy code?

RQ4 has the largest consequences. We ran the same full configuration under a deterministic mock and under a real, remote model. The two runs disagree by roughly three orders of magnitude on recovery time, and they also disagree on whether the executed mitigation was correct (Section VI-E). The mock run is not wrong in any

ordinary sense. It measures a faster and more forgiving experiment faithfully, but a governance claim drawn from that experiment does not transfer to the system an operator actually runs. The same caution applies to the other results in this paper: an evaluation that relies on the mock's near-zero latency, or on a cost model that assumes away the provisioning gap, reports gains that a live deployment does not deliver.

We count a contribution only if this campaign both implements and measures it. Four qualify.

1) A live-model evaluation of eight configurations (three non-agent comparators, the full system, and four single-agent ablations) across four injected fault scenarios. The campaign comprises 560 runs with per-run provenance and billing-accurate LLM accounting (Section V).
2) Explicit non-LLM baselines and a decision-quality metric. A rule-based responder and a threshold autoscaler, each modelled with a fixed remediation latency, run through the same campaign and show whether the agent system beats cheap automation as well as static orchestration. A correct-mitigation metric (Eq. (5)) shows whether the system acts correctly, not only quickly (Section VI).
3) An adversarial evaluation against an independent oracle. A generated corpus is graded by a specification written separately from the policy code under test, so the containment rate does not rest on the policy grading itself (Sections VI and VII-F).
4) A disclosed cost model with closed-form sensitivity analyses for the two assumptions on which the headline numbers depend: the simulated human's latency and the provisioning gap (Eq. (4)). We also report where the model's metered term falls short of a like-for-like measurement (Section VIII-D).

Two more components exist in the code but were not exercised in this campaign, and we describe them as architecture, not results. The first is a cross-model harness for testing the base paper's claim of model independence; we have not yet run its live sweep against real providers. The second is a production trust core with graduated autonomy, a durable kill switch, and a per-target blast-radius cap. All runs used one fixed mode, so we have no mode-transition or kill-switch latency to report. Section IV describes both, and Section VIII lists them as open gaps. Section VII collects production-hardening lessons, each taken from a defect the system really had.

We did not design the architecture. The three planes, the four bounded agents, the policy gate, and the observe–reason–propose loop all come from the base paper [5], and our job was to build them as described. There is no new agent design, policy language, or statistical method here. Our contribution is the evidence, the way it was measured, and a set of defects that surfaced once we checked the system against its stated guarantees instead of a test suite built on the same assumptions as the code.

Section II covers related work. Section III sets up the problem and defines the metrics, and Section IV describes what we built. The experimental design is in Section V. Section VI gives the results, with most attention on the gap between mock and live runs. The hardening lessons follow in Section VII, Section VIII covers implications, limitations, and threats to validity, and Section IX concludes.

II. BACKGROUND AND RELATED WORK

Four strands of prior work matter here: LLM agents for cloud operations, how far LLM agent evaluations can be trusted, runtime governance of agent actions, and the non-LLM automation that such agents ought to be compared with. We go through them below and close with how this study differs.

A. The Base Architecture

The base paper [5] splits the system into a data plane, a policy plane, and an agentic control plane. The control plane hosts four bounded agents, and none of them can do more than submit a validated action proposal. Its evaluation compares the system with static orchestration on a set of injected failures. We kept the same decomposition. Where the paper left a choice open, the accompanying artifact records what we chose and what else we considered.

B. Non-LLM Automation as a Comparator

Orchestrators such as Airflow, Prefect, and Dagster are built around execution. They schedule tasks, retry them, and raise alerts, but they cannot reason about a failure nobody anticipated. Simple automation is still far from weak. Google's Autopilot [9] is a heuristic autoscaler with no LLM, and it manages a large share of Google's fleet. A baseline of this kind is a fair comparator, not a strawman. Real pipelines fail for 41 documented reasons [1], and comparing an agent system only with static orchestration says nothing about whether it handles those failures better than cheap automation would. In our runs, two idealized heuristic baselines beat the full agent system on both recovery time and manual interventions (Section VI).

C. LLM Agents for Cloud Operations

An agent that watches a system and proposes actions usually follows a reasoning-and-acting loop [15]. RCACopilot [2] and RCAgent [3] use tool-augmented LLMs for root-cause analysis of production incidents at Microsoft and Alibaba Cloud, respectively. Both end with a suggested diagnosis. The system studied here goes one step further and executes the action, which is why it needs a policy gate. OpsAgent [4], a multi-agent incident-management system running at Lenovo, suggests that industry is heading the same way.

D. Evaluation Validity for LLM Agents

Our central finding is that a mock and a live evaluation of the same system diverge by orders of magnitude (Section VI-E). This finding belongs to a growing literature on the sim-to-real gap for LLM agents. The closest theoretical framing [6] decomposes the gap into observation, action, transition, and reward mismatches within a unified MDP view of foundation-model agents. Empirical studies point the same way. LLM-simulated users have been found to be "excessively cooperative" and to inflate agent success relative to grounded real-user data [7], and agent reliability drops when the same agents move from clean benchmark conditions to production-like stress such as task perturbation, tool faults, and rate limiting [8]. TheAgentCompany [16], a high-fidelity simulated workplace, finds that even its best agent completes only about 30% of consequential tasks, which identifies benchmark realism as a problem separate from the mock-versus-live question. Benchmark validity is also questioned more broadly: a survey of more than 1,300 agentic benchmark papers [17] finds that none meets basic procedural-validity criteria for public-sector deployment. Our contribution to this literature is a controlled mock-versus-live comparison of one system through one code path.

E. Graduated Autonomy and Runtime Policy

The trust core described in Section IV, with its shadow, approval, and autonomous modes, a durable kill switch, and a blast-radius cap, builds on the game-theoretic treatment of shutdown incentives [18]. A related framework [19] expresses the same idea in governance terms by separating an agent's permitted autonomy level from its capability level, and it demonstrates the separation on an enterprise data-engineering agent whose structure closely resembles the system studied here.

The policy gate runs on Open Policy Agent [20] and its Rego language. The base design's safety case is that the policy, not the model, bounds what can happen, and that case is only as strong as the policy's behavior on inputs its authors did not consider. One critique [12] argues that Rego-style languages are not expressive enough for agentic governance, and our adversarial corpus (Section VI) tests that concern empirically. ActGov [21] validates every proposed tool call against an SMT-verified policy engine before execution, which is functionally the same per-action gate that our ProposedAction contract and OPA policies implement.

F. Adversarial Evaluation

A policy bound that is asserted has not thereby been tested. Among extensible benchmarks, AgentDojo [10] and the Agent Security Bench [11], which reports attack success rates of up to 84.3% against weak defenses, are the closest to the corpus used here. The difference lies in the oracle. Ours is an independently written specification of the policy rather than the policy engine under test, so the containment figure does not depend on the policy grading itself.

G. Decision Quality and Cross-Model Generalization

Fast resolution and correct resolution are different things. AIOpsLab [22] scores cloud agents on detection, localization, and mitigation instead of latency alone. OpenSec [23] reports that frontier models are good at detecting incidents but badly calibrated: two current models had false-positive containment-action rates of 82.5% and 45%. Our correct-mitigation metric (Eq. (5), Section V) is built on the same distinction.

According to the base paper, the GPT, Claude, and Gemini model families were interchangeable as backends, yet no cross-model results are shown. Work in other settings gives reason for doubt. Sanna [13] found a 43.4-point drop in accuracy when a behavioral backdoor detector was evaluated across six production LLMs instead of in-distribution. Zhang et al. [14] gave agents identical input and still saw their behavior change with model version, open or closed weights, serving provider, and system prompt; whether the weights had been released predicted behavior better than which model it was. We built the cross-model harness of Section IV to test this claim, but did not run its live sweep in this campaign (Section VIII).

H. Fault Injection and Statistics

Deliberate fault injection is a standard way to test resilience [24]. Our seeded injector reproduces four scenarios exactly: schema drift, upstream delay, resource contention, and ingress burst. For inference we use bootstrap resampling [25], the Mann–Whitney rank-sum test [26], the Wilcoxon signed-rank test [27], Holm's step-down correction [28], Cliff's δ as an ordinal effect size [29], and Wilson score intervals [30] for proportions near one, where the normal approximation collapses to a zero-width interval.

I. Positioning

The base paper presents policy-bounded agentic control as its advantage over unconstrained LLM-driven automation. However, it asserts that the bound holds without testing it against an adversary, offers no cross-model results for its claim of model independence, and compares only against static orchestration. We test the bound adversarially, compare against explicit, idealized non-LLM baselines, and state that the harness for the cross-model claim exists but has not been run live. Prior work on the sim-to-real gap already shows that agents can fail under live or adversarial conditions. This study adds evidence from one controlled campaign that the same system, run through the same code path, reaches qualitatively different conclusions depending on whether the model in the loop is mocked or live.

III. PROBLEM FORMULATION

A. Setting

A pipeline emits a telemetry snapshot s_k at each control tick k. The tick period τ is 15 s by default and was set

to 2 s in the evaluation campaign. A fault f is injected at time t_f, detected at $t_d \geq t_f$, and resolved at $t_r \geq t_d$, either by an executed mitigation or by an escalation that the on-call engineer handles. A configuration π is a responder that maps snapshots to actions. In an agent configuration, each enabled agent g proposes an action $a = (g, \text{type}, \text{target}, \text{params}, \text{justification}, \text{confidence})$ from the typed action space $\mathcal{A}$ of the contract (Section IV). A policy gate G decides each proposal from a context $c = (m, B, n, \kappa, v)$ that the system builds outside the agent's influence. Its parts are the projected marginal cost m of the action, the budget B still left, the number n of actions the proposing agent has recorded in the past ten minutes, the schema-compatibility class κ, and a flag v that is true when a prior version exists. The gate returns allow, deny, escalate, or allow-and-notify.

B. Gate Semantics

The gate applies the rate limit of Eq. (1) before any agent-specific policy and fails toward the human:

$$G(a, c) = \text{deny} \quad \text{if } n \geq n_{\max}, \qquad n_{\max} = 5. \tag{1}$$

A proposal that matches no policy pack, or that cannot be evaluated because the policy engine is unreachable, is escalated rather than allowed. An optimization action that scales resources passes the cost policy only if the condition in Eq. (2) holds:

$$m \leq 0 \quad \text{or} \quad m \leq B. \tag{2}$$

A scale-down, whose marginal cost is non-positive, therefore always passes. Rollback is permitted only when v holds, and a schema change is permitted only when κ is backward-compatible. Section IV describes all four policy packs.

C. Metrics

Each run injects a single fault. Its recovery time, cost, and decision correctness are defined by Eqs. (3), (4), and (5):

$$\text{MTTR}(r) = t_r - t_d, \tag{3}$$

$$C(r) = C_{\text{meas}}(r) + \rho_c \, u_\pi \, T_h, \tag{4}$$

$$D(r) = \mathbf{1}\big[\text{executed mitigation} \in \mathcal{A}^*(f)\big]. \tag{5}$$

Here C_{meas} is the metered compute and storage cost, with compute priced at $\rho_c = 0.05$ per unit-second and storage at 0.01 per GB-hour; u_π is the allocation that configuration π holds over the horizon $T_h = 300$ s; and $\mathcal{A}^*(f)$ is the set of accepted mitigations for the injected fault. The three configurations that right-size their allocation, autoscale, optimization_only, and full, hold $u_\pi = 3$ units, and the others hold a static $u_\pi = 8$. Non-agent configurations never execute an agent mitigation, so $D(r)$ in Eq. (5) is 0 for them by construction.

Like the base paper, Eq. (3) measures recovery from detection. In the non-agent configurations a fixed monitor stamps detection at injection, $t_d = t_f$, so their MTTR also covers the detection interval. In an agent configuration the clock starts only when the monitoring agent flags the fault, which can be up to one tick plus one model call after injection. Manual interventions and data freshness are direct counts and lags read from telemetry.

Algorithm 1 One control tick.

```
record a heartbeat
if the kill switch is engaged then
    return
end if
run the monitoring agent: detect faults and stamp t_d
if an unresolved fault exists then
    P ← ∅
    for each enabled reactive agent g ∈ {schema, recovery, optimization} do
        a ← observe and reason (invalid output becomes no_action)
        add a to P unless it is no_action
    end for
    for each contested target, keep the proposal with the highest (agent priority, confidence)
    for each winning proposal a do
        skip if the target lock is held or the blast-radius cap is reached
        evaluate d ← G(a, c), then commit an intent row
        execute a if d allows it, or hand it to the on-call if d escalates
        update the audit row with the outcome
    end for
end if
```

D. The Evaluation-Validity Question

Let $\tilde{x}_{\text{live}}$ and $\tilde{x}_{\text{mock}}$ denote the median of metric x for the same configuration under a real language model and under a deterministic mock, respectively. We define the divergence ratio as

$$R_x = \frac{\tilde{x}_{\text{live}}}{\tilde{x}_{\text{mock}}}. \tag{6}$$

Reporting $\tilde{x}_{\text{mock}}$ as the system's performance is valid only when R_x in Eq. (6) is close to one. Section VI-E measures R_x for each metric, which answers RQ4 of Section I.

E. Threat Model

The adversary controls only its own proposal a. The context c, the policy engine, and the executor are trusted components built by the system. The safety property under test is containment: no proposal that the documented policy semantics classifies as unsafe may be allowed.

Algorithm 1 summarizes the control loop that Section IV describes in prose. Agent priority ranks recovery above schema and schema above optimization. The non-agent configurations skip the loop and resolve faults directly (Section V).

IV. SYSTEM

The system is organized in seven layers (Fig. 1). Layers 1 and 2 form the data plane and its telemetry, layer 3 holds the four bounded agents, layer 4 is the policy gate, and layer 5 executes and audits whatever the gate allows. Layers 6 and 7, orchestration and enterprise governance, extend the base paper's design. The campaign exercised layers 1 through 4, the write-ahead audit log in layer 5, and the control loop and arbitration in layer 6. It did not exercise the autonomy-mode transitions, did not engage the kill switch, and did not use layer 7. The remainder of this section describes the layers in order.

A. Layers 1 and 2: Data Plane and Telemetry

A batch pipeline in Apache Airflow ingests a TPC-DS-derived dataset into versioned warehouse partitions, and a streaming consumer reads from a Redpanda topic. All telemetry, including pipeline metrics, resource usage, task runs, agent actions, failure events, and manual interventions, is written to a PostgreSQL telemetry schema. Every experimental metric in this paper is reconstructed from that schema rather than logged separately. The same layer tracks data freshness and maintains the cost ledger behind the cost model of Eq. (4).

B. Layers 3 and 6: Agents and Orchestration

All four agents run the same loop. Each one observes a telemetry snapshot, reasons about it with a live or mock language model, proposes an action, and acts only through the gate. The monitoring agent is the detector and is enabled in every agent configuration, so recovery time remains measurable in every ablation. The optimization agent scales workers or pool slots, or reprioritizes work. The schema agent accepts compatible schema changes and contains breaking ones. The recovery agent can retry, replay, roll back, partially recompute, or hand the fault to a human. When two proposals target the same resource, a static agent-priority bid resolves the conflict. An earlier design relied on lock ordering for this purpose, but lock ordering turned out not to provide the property, so a per-target advisory lock now serves only to stop a second process from acting on the same target.

C. The Contract Between Layers 3 and 4

An agent's only output is a ProposedAction with six fields: agent, action type, target, parameters, justification, and confidence. A typed schema validates each proposal and rejects any action type outside the agent's allowlist and any scaling target that is not an integer of at least one. The second check was added after the adversarial corpus (Section VI) showed that it was missing. Output that fails validation is rejected and logged, the agent_output_invalid counter goes up, and that agent's turn becomes a no_action. Agents never emit or execute code.

D. Layer 4: Policy Gate

Each proposal carries a context built by the system: the projected marginal cost, the remaining budget, the number of actions the proposing agent has taken in the last ten minutes, the schema compatibility, and whether a prior version exists. The proposal then passes through four Rego policy packs: the rate limit of Eq. (1), the cost budget of Eq. (2), a recovery-approval policy that allows rollback only if a prior version exists, and a schema-compatibility policy. The gate answers allow, deny, escalate, or allow-and-notify. When the policy engine cannot be reached, the gate escalates; it never allows by default.

E. Layer 5: Execution and Audit

Before the executor starts, the system writes and commits an intent row. After the action, it updates that row with the outcome. If the process crashes mid-action, what remains is a durable executing row, not a missing record. The original design did not have this write-ahead ordering; we added it later. Allowed actions reach their side effects through a dispatcher that picks a handler by action type. On Airflow the handlers trigger DAG runs, clear tasks, and patch pool slots. On PostgreSQL they perform rollback, quarantine, schema mapping, and desired-state updates.

F. Simulated Human

The on-call engineer is simulated. An escalation is resolved after a seeded log-normal delay (median 360 s, $\sigma = 0.5$). We chose this delay and did not measure it, and Section VI-G shows how far the headline results move when it changes.

G. Cross-Model Harness, Implemented but Not Evaluated

The cross-model harness takes one scenario, runs it through a configurable list of LLM backends, and scores each model on decision correctness (Eq. (5)), latency, and token use. That is a direct test of the base paper's model-independence claim. The harness has unit tests, but we did not run or record a sweep across real provider models during this campaign, so this paper reports no results from it.

H. Layers 5 and 6: Trust Core, Implemented but Not Evaluated

The system also adds something the base paper does not have: a graduated-autonomy layer with three modes. In shadow mode, proposals are logged and the pipeline is left alone. In approval mode, allowed actions wait in a queue for a human to decide. In autonomous mode they run directly. Two further limits act independently of the policy: a durable kill switch that the loop checks once per tick, and an hourly blast-radius cap for each target. All three are implemented and unit-tested. Every configuration in the campaign, however, stayed in autonomous mode for the whole run, so we can describe what this layer does but not how it performed.

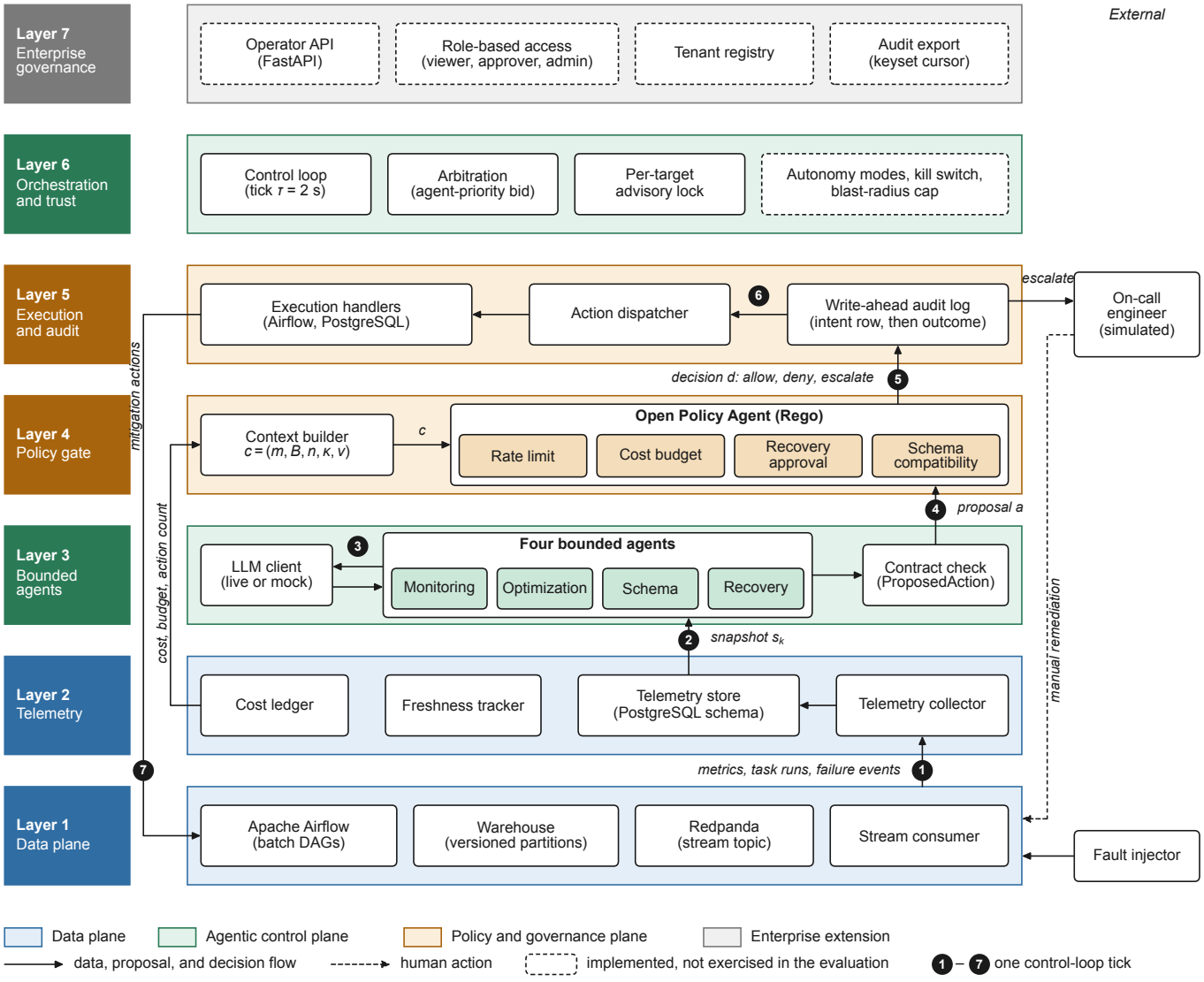


Fig. 1. Seven-layer system architecture. Shading shows which logical plane of the base design, or the enterprise extension, each layer belongs to. Numbered markers follow one control-loop tick.

I. Layer 7: Enterprise Governance, Implemented but Not Evaluated

An operator API provides read access to proposals, the audit trail, and costs, and it exports the full audit log through a keyset cursor. Role-based access control with viewer, approver, and admin roles, together with a tenant registry, binds each actor to a tenant and lets an operator suspend a tenant. This layer supports multi-tenant deployment but played no part in the experimental measurements.

J. Cloud Deployment

To show that the governance stack runs outside the laboratory host, we deployed it to a single Microsoft Azure virtual machine (Fig. 2): a Standard_B2pls_v2 instance (Arm64, 2 vCPU, 4 GB) running Ubuntu 24.04 in the Southeast Asia region. One script provisions the machine, and cloud-init installs Docker, generates every secret on the machine itself, and starts a Docker Compose stack of PostgreSQL, the policy engine, the operator API, and the control loop. Caddy sits in front as a reverse proxy, gets a certificate from Let's Encrypt, and serves the API over HTTPS. Ports 443 and 80 are open to the internet, while SSH is open only to the deployer's address. The policy engine is the same pinned release used in the campaign (Table I), built for Arm64. There is no production pipeline behind this deployment, so it runs in shadow mode with the mock model and a no-op connector: it watches and proposes, and it never acts. We checked that the certificate is valid, that HTTP redirects to HTTPS, that every protected endpoint asks for authentication, and that all seven readiness checks pass. Nothing in Section VI was measured on this machine. The campaign needs the full stack, including Airflow and Redpanda, which this deployment does not run.

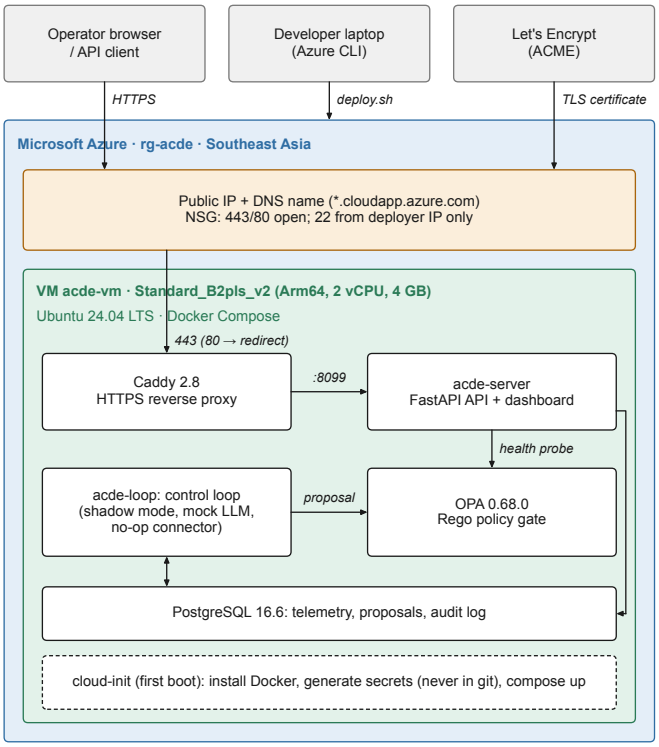


Fig. 2. Cloud deployment on Microsoft Azure.

V. EXPERIMENTAL DESIGN

A. Testbed

Everything runs in containers on one node: PostgreSQL, Redpanda, Airflow, and OPA. Hardware, software versions, and fixed parameters are listed in Table I. We pin the policy engine to one release, and Section VI explains why that matters. Runs never overlap. The resource-contention scenario puts real load on the CPU, and a second run in parallel would add its own load to the measurement.

B. Scenarios and Configurations

Four injected faults are crossed with eight configurations. The faults are schema drift (a column's type changes), upstream delay, resource contention, and an ingress burst. The configurations are baseline (static orchestration with the simulated human), rule_based (deterministic automation), autoscale (threshold autoscaling), full (all four agents), and four ablations. Three ablations add exactly one agent to monitoring: recovery_only, optimization_only, and schema_only. The fourth, monitor_only, runs monitoring alone. None of the three non-agent configurations calls a model.

C. Non-Agent Baselines

The three non-agent configurations model a responder's behavior rather than run a deployed controller. In baseline, every fault escalates to the simulated on-call engineer. In rule_based, a fault covered by the rule set (upstream delay, resource contention, or ingress burst) is resolved 30 s after detection, while schema drift, which requires reasoning, escalates to the human. In autoscale, the reactive scaler resolves resource contention and ingress bursts 20 s after detection, and every data fault escalates. The recovery times of these configurations therefore encode two assumed remediation latencies; they are not measurements of a real rule engine or autoscaler. These comparators are idealized. How fast they are comes from the latencies we assumed, and any fault they do not cover still goes to the human, which caps how well they can do.

D. Matrix and Arms

The baselines and full have 80 runs each, pooled over the four scenarios. The ablations have fewer, and Table II lists the realized n for every configuration. Only agent configurations call a model, so we split the campaign into three arms. Each arm can be resumed on its own and writes to its own result directory, which keeps live and mock runs from being pooled by mistake. Arm A contains the agent configurations with the live model. Arm B contains the non-agent configurations with the model switched off; since they never call it, their results would be the same under live and mock. Arm C runs full with a deterministic mock, as the direct counterpart of full in arm A. The arms hold 240, 240, and 80 runs, 560 altogether. Every run begins with a 120 s warm-up before the fault is injected and ends with a 5 s settle. Agent runs execute a 300 s control loop in between, whereas the non-agent responders resolve the fault without a loop.

E. Live Model and Accounting

The live arm calls a remote language model at temperature zero through an OpenAI-compatible endpoint. The reasoning agents share one model, and monitoring uses a smaller, faster one. Each run manifest records the model identifiers, and a per-run budget limits both the number of model calls and the token count.

Two details of the accounting affect validity. First, a proposal for a snapshot that has already been seen is served from an in-run cache, so summing per-action token columns would count one real call several times. For that reason we count real calls, cache hits, degraded proposals, invalid outputs, tokens, and latency at the point where a call is made or skipped. A proposal is counted as degraded if the budget has run out, if the provider is unavailable, or if it replays an earlier degradation. Second, the project's pilot showed that caching a transient provider failure lets a single rate-limit episode degrade every later identical snapshot (Section VII). Transient failures are therefore no longer cached, and the degraded count is tracked separately.

F. Metrics

We report the five metrics defined in Section III: mean time to recovery (MTTR, Eq. (3)), operational cost under

TABLE I
Experimental Testbed and Fixed Design Parameters

Item	Setting
Host	One Apple M2 machine (arm64), 8 cores, 8 GB memory, macOS 15.7.2; runs are strictly sequential
Software	Docker 29.6.1; PostgreSQL 16.6; Redpanda v24.2.18; Apache Airflow 2.10.5 (Python 3.11); OPA 0.68.0
Batch data	Downscaled TPC-DS-derived synthetic fact table, 20,000 rows
Live model	Reasoning agents: nvidia/nemotron-3-ultra-550b-a55b; monitoring: nvidia/nemotron-3-super-120b-a12b; hosted OpenAI-compatible endpoint, temperature 0
LLM budget	60 calls and 150,000 tokens per run
Timing	120 s warm-up, 300 s control loop, 5 s settle; control tick 2 s (default 15 s)
Human model	Log-normal delay, median 360 s, $\sigma = 0.5$
Cost model	Static allocation 8 units, right-sized 3 units, horizon 300 s; 0.05 per compute unit-second, 0.01 per GB-hour
Baseline latencies	rule_based resolves covered faults 30 s after detection; autoscale 20 s after detection
Policy limits	Rate limit of 5 actions per 10 minutes; default cost budget of 100 units per run

a disclosed model (Eq. (4)), manual interventions, data freshness, and decision correctness (Eq. (5)). Manual interventions are counted as rows in a telemetry table that all configurations write to. Freshness is a lag in seconds, taken from the same telemetry.

G. Seeds and Provenance

The seed of each run is sha256(config:scenario:replicate) mod 2^{32}. Since the configuration name goes into the hash, two runs with the same (scenario, replicate) but different configurations get different fault and human-latency randomness. The matching is therefore nominal rather than paired, so we treat the samples as independent and use the rank-sum test as the primary test; the signed-rank test on the matched index is kept only for continuity with earlier analysis. Every manifest line records the profile, the model mode, the model identifiers (credentials are never written), the timings, the human-latency and provisioning parameters, the budgets, the code revision, and a flag for uncommitted changes to the experiment code. The analysis refuses to merge runs with differing provenance unless an explicit, documented override is given.

H. Statistics

We report medians with interquartile ranges, together with bootstrap 95% intervals for medians and for differences between medians. Each Cliff's δ is reported with its own bootstrap interval and read against the conventional thresholds: $|\delta| < 0.147$ negligible, 0.147–0.33 small, 0.33–0.474 medium, and ≥ 0.474 large [29]. Mann–Whitney U tests are Holm-corrected across the full family of comparisons, and proportions are given with Wilson intervals. All comparisons in Section VI are exploratory. The campaign was designed and run before any individual comparison was written up, and it was not pre-registered as a confirmatory study.

I. Adversarial Evaluation

A generated corpus probes the policy layer directly. It covers every legal (agent, action) pair on an exhaustive grid, crossed with the context values that drive decisions. Boundary cases are included, for example 4, 5, and 6 recent actions, or a cost exactly at or just above the budget. The corpus also has extreme parameter values chosen by an attacker and priced by the real gate, hostile strings in free-text fields, illegal proposals sent straight to the policy engine so that they skip the contract, and a seeded fuzz. Expected verdicts come from a separate Python implementation of the documented policy semantics, which shares no code with the Rego being tested. Both were written by the same team, so the independence is in the code, not in the people. In line with the threat model, the agent controls its own proposal and nothing else. The system builds the policy context, and the agent cannot reach it.

J. Sensitivity

The sampled human latency is median $\cdot\, e^{\sigma Z}$, where the seed fixes Z. Under this scale family, any outcome handled entirely by the human grows in exact proportion to the assumed median. We checked that linearity against the sampler itself and used it to compute, in closed form, the human median below which each configuration no longer beats the baseline. For cost, we sweep the provisioning parameters and report the break-even point.

VI. RESULTS

All measured values in this section come straight from the campaign's raw telemetry (Section V), and each can be traced to the script that computed it. The only numbers typed by hand are design constants and figures quoted from the base paper. All 560 runs across the three arms completed; none aborted and none hit the token ceiling. The provenance check flagged 1 deviation across the merged arms. Midway through the campaign the supervisor had to be restarted after a host infrastructure failure that had nothing to do with the experiment. The

fix shipped with that restart touched only the supervisor's health check and no code that a run executes, and we confirmed this with a diff before treating the three arms as one dataset. The first three subsections answer RQ1–RQ3 with the live model. Section VI-E then shows that those answers change depending on whether the model was real (RQ4).

A. Does the Full Agent System Beat Static Orchestration?

Table II reports medians and interquartile ranges per configuration, and Table III compares each configuration with baseline using bootstrap confidence intervals, Cliff's δ, and Holm-corrected Mann–Whitney p-values. Figures 3 and 4 show the underlying distributions and effect sizes. The full configuration changes MTTR (Eq. (3)) from 378 s to 235 s ($\Delta = -143$ s, -38%, Cliff's $\delta = -0.56$, $p_{\text{Holm}} < 0.001$) and cost from 158 to 69.8 units (-56%, $\delta = -0.66$, $p_{\text{Holm}} < 0.001$). Both effects are significant, but the MTTR reduction is far smaller than the one the mock evaluation of the same configuration produces (Section VI-E). Most of it is also an artifact of the simulated engineer, which rewarded repeated escalation (Section VII-H). This caveat applies to every MTTR comparison that follows.

Cost is the only headline metric whose change reaches the base paper's reported 25% reduction (Table III), but the reduction depends on the assumed provisioning gap (Section VI-G). Most of it comes from the provisioning term of Eq. (4), which is an assumption, rather than from metered compute. The pooled figure also hides two features of the cost data. First, the saving depends on the scenario: the median cost of full is below that of baseline in three scenarios but above it in ingress_burst (523 against 158 units). Second, agent configurations that do not right-size cost far more than baseline in the pooled data: monitor_only and recovery_only by 676% and 676%, and schema_only by 484% (Table III). Because these configurations hold the same static allocation as baseline, the excess lies entirely in the metered compute term, and it is largely an artifact of how that term is measured (Section VIII-D). Resources are sampled once before injection and once after the settle, so an agent configuration meters compute over its full 300 s loop, whereas a non-agent configuration, whose response completes without waiting, meters only a few seconds. The saving is therefore better attributed to the right-sizing configurations, chiefly the optimization agent, than to agent control in general.

Manual interventions move in the opposite direction (Tables II and III). The median rises from 1.00 to 6.00 ($\Delta = 5.00$, 500%, $\delta = 0.79$, $p_{\text{Holm}} < 0.001$), the reverse of the base paper's reported reduction of more than 70%, and recovery_only shows the same rise (500%). One plausible mechanism is that baseline escalates exactly once per injected fault by construction, whereas an agent configuration that is rate-limited or denied by the budget can propose again on a later tick of the same 300 s loop, with each cycle adding an escalation. We did not instrument which mechanism dominates, so we report the measurement and leave the mechanism open (Section VIII). Because reducing manual interventions is the system's stated purpose, we count this increase as a negative result for the design. The answer to RQ1 is therefore mixed: with a live model, the full system beats static orchestration on recovery time and cost, but not on manual interventions.

B. Is the Full System Better Than Cheap, Non-Agent Automation?

The rule_based and autoscale configurations call no model, yet both beat every agent configuration on MTTR, including full (Tables II and III). That ranking follows from the remediation latencies we modelled in Section V. Each comparator resolves a covered fault after a fixed 30 s or 20 s, so its numbers show what an idealized non-LLM responder could do, not how fast a deployed one actually is. rule_based reaches 30.0 s (-92% relative to baseline, $\delta = -0.73$), and autoscale reaches 52.4 s (-86%, $\delta = -0.49$), compared with -38% for full. Neither raises manual interventions (-100% and -50%), while full raises them.

On cost (Table III), autoscale moves the median by -48%, less than the -56% achieved by full. rule_based, which does not right-size, shows no cost effect (0%, $p_{\text{Holm}} = 1.000$), as its design predicts. By construction, neither comparator can execute an accepted mitigation (decision_correct= 0), and decision quality is the one axis on which full wins outright (Section VI-C). For these fault scenarios and this control-loop timing, cheap deterministic automation beats the LLM-driven system on two of the four metrics. For idealized comparators, then, the answer to RQ2 is negative: the agent system does not beat cheap automation on speed or on manual interventions.

C. Decision Quality

The decision_correct metric is scored against a per-scenario set of accepted mitigations (Eq. (5), Section V), and every non-agent configuration scores zero by construction. Among the agent configurations (Table III), only full ($\delta = 0.42$, $p_{\text{Holm}} < 0.001$), optimization_only ($\delta = 0.25$), and schema_only ($\delta = 0.25$) change the metric significantly; monitor_only and recovery_only do not. Against an all-zero comparator, Cliff's δ equals the treatment group's success rate, so these values are also correct-mitigation rates in the sense of Eq. (5): full executes an accepted mitigation in 0.42 of its runs. This is the one metric on which the live system achieves something that neither the static baseline nor the rule-based comparator can achieve at all. It answers RQ3 only in part, however, because fewer than half of the full system's runs act correctly. As Section VI-E shows, the mock evaluation scores the same configuration at 1.00.

D. Per-Scenario Heterogeneity

Table IV and Fig. 5 break MTTR down by configuration and scenario. No single configuration dominates every fault

TABLE II
Median [IQR] for Each Configuration; n Counts Replicates Pooled Over All Scenarios

Config	LLM	n	MTTR (s)	Cost	Manual interv.	Decision correct
baseline	none	80	378 [244]	158 [113]	1.00 [0]	0 [0]
rule_based	none	80	30.0 [15.9]	158 [113]	0 [0.250]	0 [0]
autoscale	none	80	52.4 [359]	82.5 [122]	0.500 [1.00]	0 [0]
monitor_only	live	40	359 [210]	1,223 [525]	1.00 [1.00]	0 [0]
recovery_only	live	40	220 [75.0]	1,223 [630]	6.00 [1.00]	0 [0]
optimization_only	live	40	267 [360]	76.5 [128]	1.00 [1.00]	0 [0.250]
schema_only	live	40	315 [353]	920 [122]	1.00 [1.25]	0 [0.250]
full	live	80	235 [143]	69.8 [35.1]	6.00 [8.00]	0 [1.00]

TABLE III
Each Configuration Compared With baseline: Median Difference and Cliff's δ, Each With a Bootstrap 95% CI, and the Mann–Whitney p-Value After Holm Correction ($^*p_{\text{Holm}} < 0.05$)

Config	Metric	Δ median [95% CI]	Cliff's δ [95% CI]	p_{Holm}
rule_based	MTTR (s)	−348 [−419, −303]	−0.73 [−0.85, −0.60]	<0.001*
autoscale	MTTR (s)	−326 [−406, −82.7]	−0.49 [−0.63, −0.32]	<0.001*
monitor_only	MTTR (s)	−19.2 [−99.6, 59.8]	−0.09 [−0.32, 0.13]	1.000
recovery_only	MTTR (s)	−159 [−229, −105]	−0.70 [−0.83, −0.55]	<0.001*
optimization_only	MTTR (s)	−111 [−188, 15.9]	−0.27 [−0.50, −0.025]	0.134
schema_only	MTTR (s)	−63.1 [−182, 87.4]	−0.19 [−0.42, 0.056]	0.638
full	MTTR (s)	−143 [−212, −94.7]	−0.56 [−0.70, −0.42]	<0.001*
rule_based	Cost (units)	0 [0, 0]	0.00 [−0.14, 0.14]	1.000
autoscale	Cost (units)	−75.0 [−75.0, −75.0]	−0.62 [−0.76, −0.45]	<0.001*
monitor_only	Cost (units)	1,065 [1,065, 1,074]	1.00 [1.0, 1.0]	<0.001*
recovery_only	Cost (units)	1,065 [1,065, 1,389]	1.00 [1.0, 1.0]	<0.001*
optimization_only	Cost (units)	−81.0 [−81.0, −48.3]	−0.47 [−0.72, −0.20]	<0.001*
schema_only	Cost (units)	763 [750, 763]	1.00 [1.0, 1.0]	<0.001*
full	Cost (units)	−87.7 [−96.2, −87.7]	−0.66 [−0.81, −0.51]	<0.001*
rule_based	Manual interv.	−1.00 [−1.00, −1.00]	−0.75 [−0.85, −0.65]	<0.001*
autoscale	Manual interv.	−0.500 [−1.00, 0]	−0.50 [−0.61, −0.39]	<0.001*
monitor_only	Manual interv.	0 [0, 0]	0.28 [0.15, 0.42]	<0.001*
recovery_only	Manual interv.	5.00 [5.00, 5.50]	0.97 [0.93, 1.0]	<0.001*
optimization_only	Manual interv.	0 [0, 1.00]	0.20 [−0.050, 0.42]	0.121
schema_only	Manual interv.	0 [0, 0]	0.25 [0.12, 0.38]	<0.001*
full	Manual interv.	5.00 [5.00, 8.00]	0.79 [0.70, 0.88]	<0.001*
rule_based	Decision correct	0 [0, 0]	0.00 [0, 0]	1.000
autoscale	Decision correct	0 [0, 0]	0.00 [0, 0]	1.000
monitor_only	Decision correct	0 [0, 0]	0.00 [0, 0]	1.000
recovery_only	Decision correct	0 [0, 0]	0.00 [0, 0]	1.000
optimization_only	Decision correct	0 [0, 0]	0.25 [0.12, 0.38]	<0.001*
schema_only	Decision correct	0 [0, 0]	0.25 [0.12, 0.38]	<0.001*
full	Decision correct	0 [0, 1.00]	0.42 [0.31, 0.54]	<0.001*

type. optimization_only is the fastest configuration on ingress_burst but is slower than baseline on schema_drift, a fault on which its only non-monitoring agent has no lever, and schema_only shows the mirror-image pattern. This is the pattern a single-agent ablation should produce: the agent helps where it owns the fault and is inert, or pure overhead, elsewhere. It also cautions against treating any single scenario's result as a configuration's typical behavior.

E. Mock Versus Live: The Central Finding

Table V and Fig. 6 compare full under two conditions that differ only in the model: arm A (live) and arm C, its deterministic-mock counterpart. MTTR is 235 s with the live model and 0.193 s with the mock, a divergence ratio of $R_{\text{MTTR}} = 1219$ (Eq. (6)). The median decision-correct score goes from 0 live to 1.00 mock, even though the live correct-mitigation rate is 0.42. Manual interventions drop from 6.00 to 0. Freshness drops from 14.0 s to 0.0801 s.

Cost goes the opposite way. The live arm costs less than the mock arm (69.8 vs. 833 units, Table V), and the mock even costs more than baseline. We think most of this gap comes from the harness, not the model. The campaign never reset two global settings between runs, the Airflow pool size and the streaming-worker target, so every run started from whatever the previous agent action had left behind. The mock arm ran last, right after schema_only. In schema drift and upstream delay, the mock's rule-based optimization agent does nothing, and mock full meters exactly the compute that live schema_only did. Live full ran first, from another starting state, and its metered compute stayed low in three of the four scenarios. The cost difference between the two arms therefore does not cleanly measure what the model changes, and we do not read it that way.

These results answer RQ4. The mock evaluation is not a noisier or scaled-down version of the live one. On recovery time and decision quality it is a categorically different and far more flattering experiment, and the earlier results in

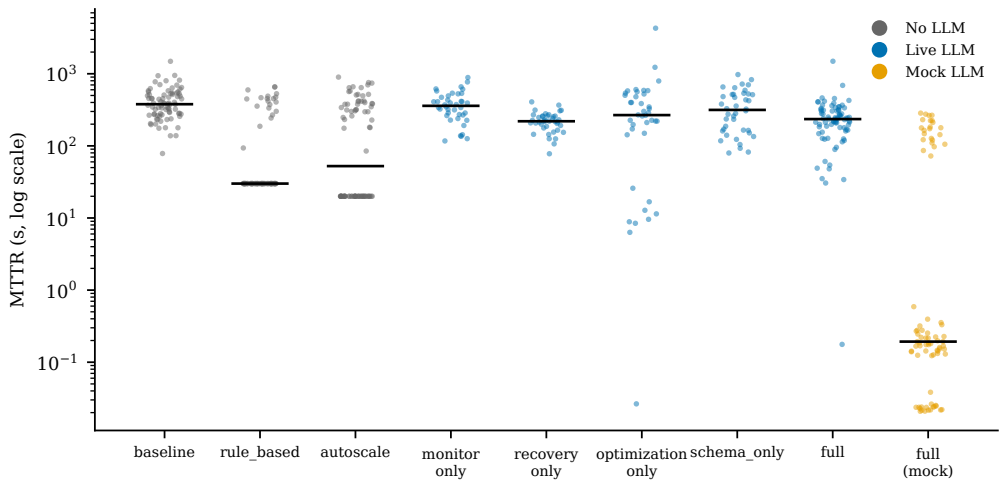


Fig. 3. MTTR of every run, grouped by configuration and plotted on a log scale.

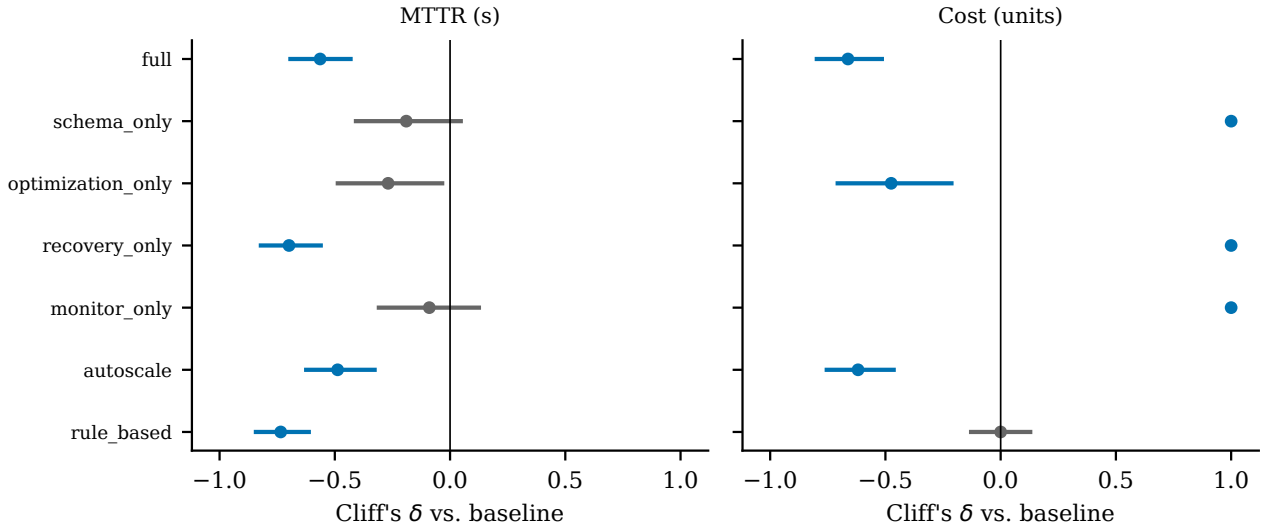


Fig. 4. Cliff's δ of each configuration against baseline for MTTR and cost. Bars show bootstrap 95% intervals.

this section should be read with that in mind. We cannot tell which evaluation path produced the base paper's headline figures, because the paper does not say. What this campaign does show, for one system, is how much the evaluation path by itself can shift the result. A claim about this class of system drawn from a mock run is a claim about the mock, not about the model it replaces.

F. LLM Accounting

Table VI reports real model calls, cache hits, degraded proposals, invalid outputs, tokens, and latency for each agent configuration, counted at the moment a call is or is not made (Section V). Over the whole campaign, the degraded and invalid rates are both zero. No run used up its budget or ran into a provider outage, and no output that failed the schema made it into the final runs. That holds for this campaign and is not guaranteed in general. In the pilot that came before it we found a live cache-poisoning defect, which is precisely what the accounting is meant to catch, and we fixed it before the campaign started.

In absolute terms, the agents cost little. Across the 240 live-arm runs, the agents made 703 real model calls and consumed 975,570 API tokens in total, and no run came close to the per-run budget: the largest used 16,042 tokens against a cap of 150,000, and at most 11 calls against a cap of 60. The per-run model latency in Table VI is small compared with the 300 s control loop, so inference time alone does not explain the live system's recovery time. We do not convert tokens into a monetary cost, because the provider's per-token prices were not recorded with the runs; token counts are the portable measure.

TABLE IV
Median MTTR (s) by Configuration and Scenario

Config	ingress_burst	resource_contention	schema_drift	upstream_delay
baseline	352	386	424	353
rule_based	30.0	30.0	433	30.0
autoscale	20.0	20.0	363	393
monitor_only	366	412	320	275
recovery_only	223	183	224	248
optimization_only	10.5	267	502	281
schema_only	453	343	120	553
full	148	318	214	237

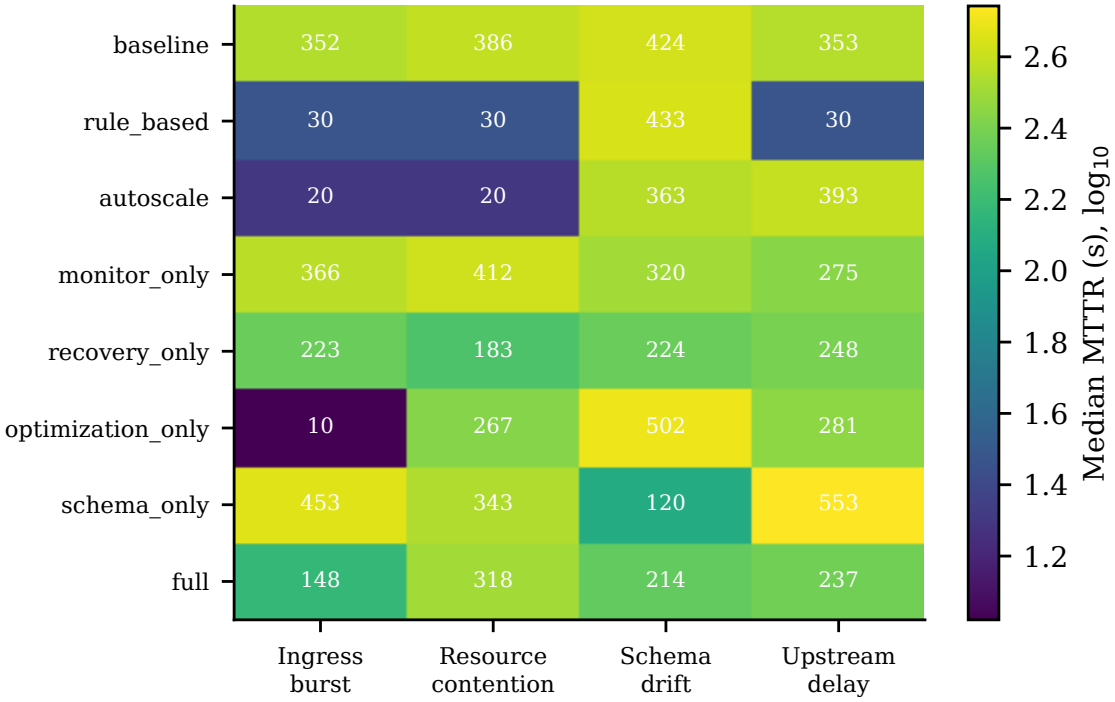


Fig. 5. MTTR by configuration and scenario.

G. Sensitivity

Table VII and Figs. 7 and 8 present the two closed-form sensitivity analyses of Section V. full continues to beat the simulated human down to a median human latency of 224 s, well below the assumed 360 s but far above the 28.5 s break-even of rule_based. The rule-based responder's speed advantage therefore survives a much faster on-call engineer than that of full. On cost, the reduction of full holds down to a static provisioning level of 2.15 units; below that level, a baseline that was never heavily over-provisioned removes the saving on its own. Both analyses make the point of Section VI-E from another angle: the headline numbers depend on modeling choices, so we report each number together with the condition it depends on.

H. Adversarial Containment

Table VIII grades the generated corpus (Section V) against the independent specification oracle, by category and overall, and answers RQ5. Overall containment is 3867/3867 ($\geq$ 0.9990, Wilson 95%). The over-block rate on legal proposals is zero, with the exact denominator given in the table. The policy agrees with the oracle on all 5547/5547 graded cases, with 0 gate errors. These figures were measured after the two contract-layer fixes described in lesson L6 (Section VII-F), which an earlier pass of the same corpus made necessary. They therefore describe the fixed system, and the corpus that exposed the gaps is now a committed regression test for this claim.

VII. PRODUCTION-HARDENING LESSONS

The central finding of Section VI-E has a less statistical counterpart. The system had several defects that its own test suite could not see, for the same reason that the mock

TABLE V
Live Model Versus Deterministic Mock for full Under Otherwise Identical Configuration and Scenarios, With Cliff's δ (95% CI) and Holm-Corrected p

Metric	Live	Mock	Cliff's δ [95% CI]	p_{Holm}
MTTR (s)	235	0.193	0.81 [0.72, 0.90]	<0.001
Cost (units)	69.8	833	−0.82 [−0.92, −0.71]	<0.001
Manual interv.	6.00	0	0.87 [0.79, 0.93]	<0.001
Decision correct	0	1.00	−0.57 [−0.69, −0.46]	<0.001
Freshness (s)	14.0	0.0801	0.25 [0.074, 0.42]	0.004

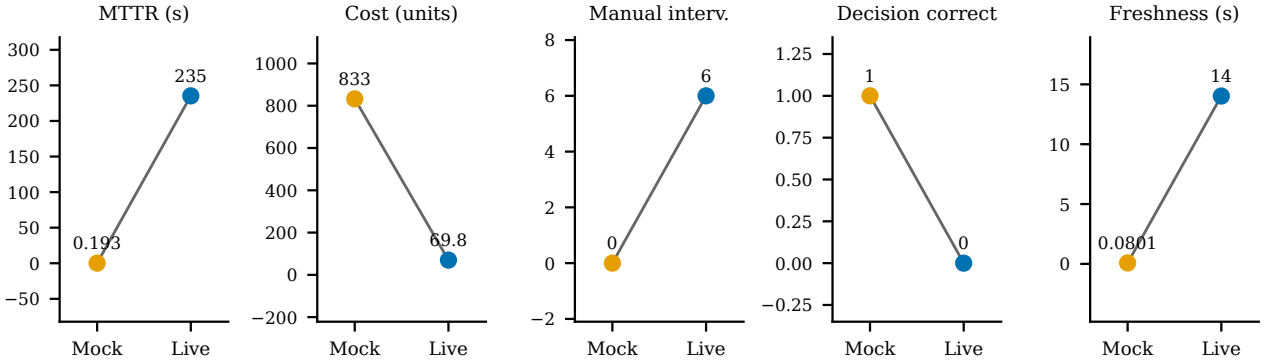


Fig. 6. Live versus mock medians for full, per metric. We use linear axes since two of the metrics are exactly zero in one arm and could not be drawn on a log axis.

TABLE VI
Model Usage per Agent Configuration, Counted Where Each Call Is Made or Skipped

Config	Calls	Cache hits	Degraded	Invalid	API tokens	Latency (s)	Degraded %
monitor_only	1.00	143	0	0	1,244	4.37	0%
recovery_only	2.00	262	0	0	2,737	10.5	0%
optimization_only	2.00	258	0	0	2,998	9.22	0%
schema_only	2.00	274	0	0	2,550	5.73	0%
full	4.00	214	0	0	5,946	15.0	0%

evaluation flatters the headline numbers: the path the tests exercised was not the path production takes. Each lesson below was found through an audit or an experiment rather than through a failing test, and each is documented in the accompanying artifact.

A. L1: The Shared Resource Nobody Declared Concurrency-Unsafe

The base design states that execution is deterministic and auditable. Versioned partition creation, however, read the current maximum version and then dropped, created, and registered a table in separate statements without a lock, so two concurrent callers targeting the same partition could compute the same next version and race. It first showed up as a continuous-integration failure that came and went. We tracked it down instead of retrying until it passed, and found that a replay by the recovery agent had triggered the same DAG that an unrelated test was using. The fix wraps the read-modify-write in a database advisory lock. Lesson: as soon as an agent can start real work, every shared resource it touches becomes a concurrency problem, and a test with a single actor will never exercise it.

B. L2: An Action Could Be Lost From the Audit Trail

At first the audit row was written after execution. If the process crashed, was killed for running out of memory, or lost its database in that window, an action could take effect and leave no record behind. The control loop swallows per-tick exceptions, so nobody would have noticed. The system's main promise is that every action passes the policy gate and is audited, and an action with no audit row breaks that promise outright. Now the intent row, which already holds the policy verdict, is committed before execution and updated with the outcome once the action finishes. Lesson: write down the intent first and act second.

C. L3: The Detector That Had No Callers

Only the fault injector wrote to the table that both the recovery-time statistics and decision scoring read from.

TABLE VII
Closed-Form Break-Even Points: The Assumed Median Human Latency (s) and Static Provisioning Level (Units) Below Which Each Configuration No Longer Beats Its Comparator

Config	MTTR (s)	Human break-even (s)	Cost reduction (%)	Static break-even
autoscale	52.4	49.9	47.6	3.00
full	235	224	55.7	2.15
monitor_only	359	342	−676	–
optimization_only	267	254	51.4	2.60
recovery_only	220	209	−676	–
rule_based	30.0	28.5	0	–
schema_only	315	300	−484	–

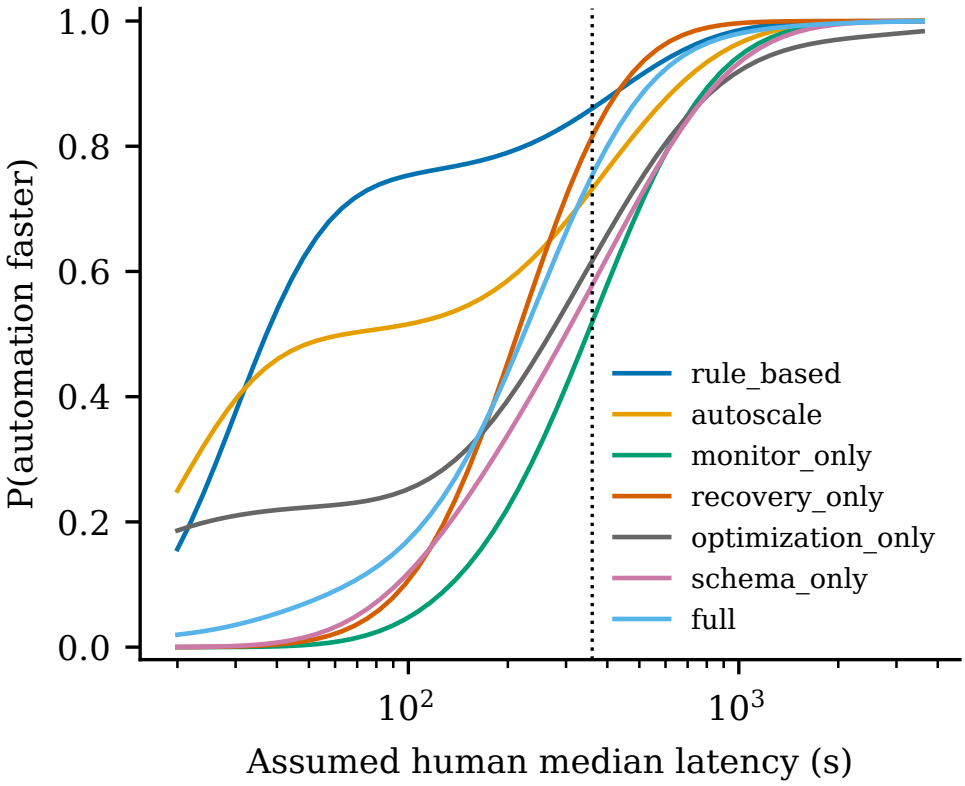


Fig. 7. Probability that automation recovers faster than the simulated human, as a function of the assumed human median latency.

The monitoring agent updated the detection time on a row that already existed but never inserted one. The result was a complete, deterministic anomaly detector, with unit tests, that nothing called. Without the injector, a real incident that the agent detected would have reached the audit table but never become an incident record, and production would have been unable to reproduce the experiment's own metrics. We noticed this while planning a later feature, by reading the code; no test failed. Lesson: a component can pass all of its own tests and still be disconnected, so follow the data from production input all the way to the reported metric.

D. L4: Two Taxonomies That Never Met

Decision correctness is scored against a set of accepted mitigations (Eq. (5)), and that set is keyed by the name of the injected scenario. Live incidents use different fault-type names. Applying the set to live incidents would have marked every real incident "incorrect by construction." A mapping from live fault types onto the same accepted-mitigation set was added before the metric was exposed outside the experiment harness. Lesson: a metric defined in the experiment's own vocabulary cannot move into production without an explicit mapping.

E. L5: Hermeticity by Enumeration Does Not Scale

One class of unit-test failure kept recurring: a test that did not mock one module's database handle turned a millisecond test into a thirty-second connection-pool timeout. We patched it one test at a time, four times over, before dealing with the cause. Now every such call goes

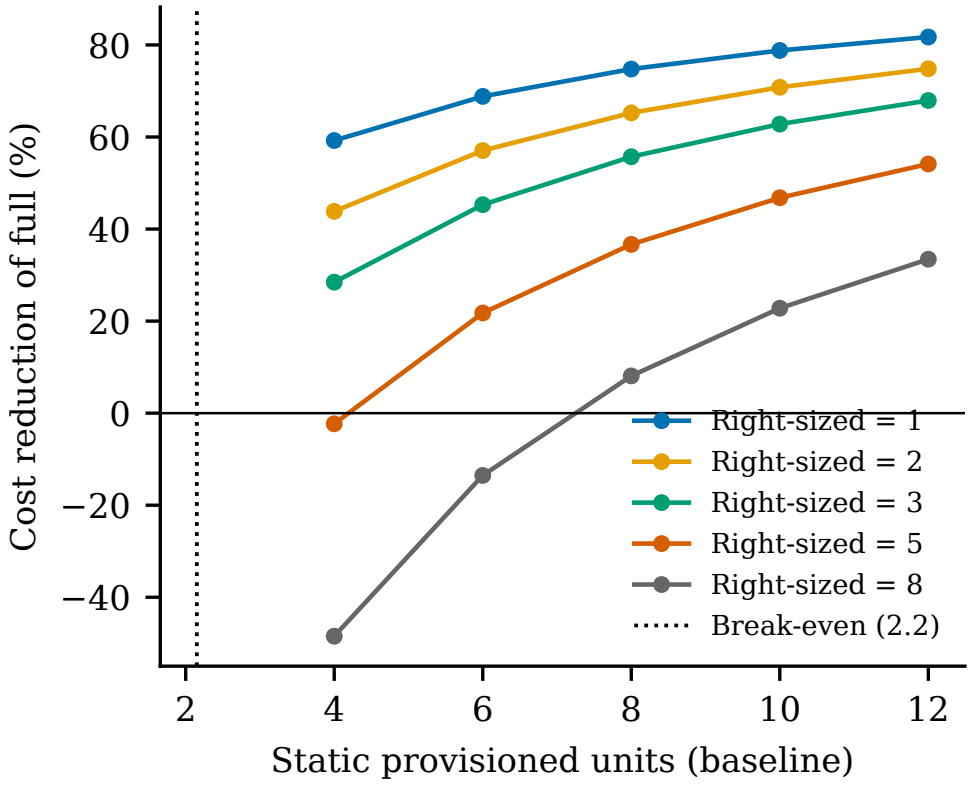


Fig. 8. Cost reduction of full as a function of the assumed static provisioning level.

TABLE VIII
Adversarial Corpus Graded Against the Independent Specification Oracle, by Category

Category	Cases	Containment [Wilson 95%]	Over-block	Oracle agreement	Fail-open
boundary_grid	4320	3056/3056 [0.9987, 1.0000]	0/1264 [0.0000, 0.0030]	4320/4320 [0.9991, 1.0000]	0
defense_in_depth	133	133/133 [0.9719, 1.0000]	–	133/133 [0.9719, 1.0000]	0
fuzz	1000	606/606 [0.9937, 1.0000]	0/394 [0.0000, 0.0097]	1000/1000 [0.9962, 1.0000]	0
injection	28	28/28 [0.8794, 1.0000]	–	28/28 [0.8794, 1.0000]	0
param_extremes	66	44/44 [0.9197, 1.0000]	0/22 [0.0000, 0.1487]	66/66 [0.9450, 1.0000]	0
Overall	5547	3867/3867 [0.9990, 1.0000]	0/1680 [0.0000, 0.0023]	5547/5547 [0.9993, 1.0000]	0

through one connection-pool accessor, and under test that accessor is patched to fail immediately. Mutation testing of this guard, by removing one essential mock, made six tests fail within seconds, each naming the exact cause. Lesson: by the fourth time the same bug turns up, stop fixing instances and fix the mechanism.

F. L6: What the Adversarial Corpus Found

Grading a generated corpus against an independent oracle revealed three things (numbers in Section VI). First, the contract layer had two gaps. The cost policy priced a scaling target against the budget but did not check its floor. Zero workers, which halts ingestion entirely, was therefore within budget, and so was a negative "scale-down." In addition, non-numeric targets raised an exception inside the gate before the audit row was written, so a rejected proposal could leave no record. Both gaps are now closed in the contract layer: scaling targets must be integers of at least one, and any violation follows the ordinary invalid-output path.

Second, under a newer policy-engine release than the one this project pins (Table I), some floating-point budget comparisons evaluated as within budget when they were not, whereas the pinned release behaved correctly. We have not traced the root cause or filed an upstream report. Whatever the cause, the practical consequence is the same: the version pin is necessary, and the corpus now doubles as an integration test that would flag any future upgrade that changes these semantics.

Third, with the contract fixes in place, the policy matched the independent specification on every graded case. Containment is now something we measured, with a confidence interval, and not a claim copied from the code under test. Lesson: when a policy's own authors say it is safe, treat that as a hypothesis until an independent oracle has tried and failed to break it.

TABLE IX
Human-Resolved Runs ("Human"), Their Median Escalations and MTTR (s), and the Median Fastest of That Many Simulated-Human Draws (s)

Config	n	Human	Escal.	MTTR	Fastest
baseline	80	80	1	378	360
rule_based	80	20	1	433	360
autoscale	80	40	1	380	360
monitor_only	40	40	1	359	360
recovery_only	40	40	6	220	195
optimization_only	40	31	2	347	274
schema_only	40	40	1	315	360
full	80	73	6	241	195

G. L7: The Pilot Found a Retired Model

Before the live campaign, an eight-run pilot at the real paper timings measured cost and wall time. Within minutes it found that the provider had retired the small model configured for the monitoring agent (HTTP 410, end of life). Because monitoring is the detector in every agent configuration, the full campaign would otherwise have run with a detector degraded to a no-op, and every downstream number would have measured the provider's retirement of the model rather than the system. The supervisor has a guard against streaks of degraded calls, so it would have caught the problem eventually, but only after several runs had been wasted. The pilot caught it on the first call. Lesson: in a long experiment that costs money, the cheapest validity check is a short run with the real settings. A model that a provider still lists may no longer answer.

H. L8: The Simulated Engineer Rewarded Repeated Escalation

After the campaign we found a flaw in the on-call model that favours any configuration escalating more than once. The simulator gave each escalation row its own log-normal delay and closed the fault when the fastest one finished. An agent that escalated on several ticks of the same fault thus drew several delays and kept the shortest.

Table IX shows the effect. We count a run as human-resolved when its MTTR is at least 60 s; the chance that even the fastest of 6 draws falls below that is about 0.001. The human resolved 73 of the 80 full runs, and those runs escalated a median of 6 times. Their median MTTR, 241 s, is much closer to the 195 s expected from the fastest of 6 draws than to the 360 s of a single draw. recovery_only shows the same pattern (220 s), whereas monitor_only, which escalates once, stays at 359 s, close to baseline. For full and recovery_only, then, the simulator rather than the agents produced most of the MTTR reduction reported in Section VI. It also explains part of the rise in manual interventions, because every re-escalation adds a row.

The simulator now pages the engineer once per incident, starting at the first escalation, with a single delay per fault that paired runs share (D-106 in the repository). We plan a pre-registered re-run with this fix; this paper reports none of its numbers. Lesson: a simulated human belongs to the measuring instrument and needs the same scrutiny as the system under test. Its output should not change with how often the system asks for help.

VIII. DISCUSSION

A. Implications for Evaluating Governed Agents

The gap between mock and live runs (Section VI-E) has direct consequences for how governed-agent systems are evaluated. Together with the lessons of Section VII, it points to four practices. First, say whether the model in the loop was real or mocked, and when both numbers exist, report both; a mock figure tells you about the mock. Second, include cheap non-LLM baselines. Here, two idealized ones beat the full agent system on recovery time and manual interventions, and a comparison with static orchestration alone would have hidden that. Third, grade safety claims against an oracle that shares no code with the mechanism being tested. Ours found two contract-layer gaps that the system's own tests had missed. Fourth, before a long campaign, pay for a short pilot at the real settings. Ours hit a retired model on its very first call.

B. Deployment Implications

For deployment, the most relevant decision-quality figure is 0.42 (Table III), not the headline MTTR: at these settings full executes an accepted mitigation in fewer than half of its runs. Together with the rise in manual interventions, this campaign gives no support for running this class of system unattended and fully autonomously against real infrastructure. The results instead favor the graduated trust ladder of Section IV as the safer production default: shadow, approval, and autonomous modes, with a durable kill switch and a per-target action cap. They also argue against treating the research benchmark's default of full autonomy as a deployment recommendation. For now, the trust core is an architectural response to a question that our measurements raise; we have not yet measured whether it works.

C. The Manual-Intervention Mechanism Is Measured, Not Explained

Section VI shows that full and recovery_only raise manual interventions compared with baseline, which contradicts the base paper. The measurement is straightforward: the metric counts rows in the telemetry.manual_interventions table, and every configuration writes to that table. Why the count rises is a separate question, and we have not answered it. One fault that stays open could escalate again on tick after tick. Repeated denials could end in hand-offs to the human. Both could be happening at once. Telling these apart needs a per-tick trace of proposal outcomes across the life of a fault, and the current telemetry does not keep one. Section VII-H settles one piece: the median human-resolved full run escalated 6 times (Table IX), so repeated escalation on an open fault is at least part of the mechanism. We leave the rest open and do not try to infer it from the total. Readers should not assume it resolves in the system's favor.

D. Limitations and Threats to Validity

1) Threats to Validity: We see nine threats. Simulated human. All comparisons with baseline depend on the log-normal on-call model. Section VI-G gives the break-even point; it does not argue that the model is right. Per-escalation human draws. The simulator drew one delay per escalation and kept the fastest, so configurations that escalate repeatedly get shorter MTTRs. Section VII-H attributes most of the MTTR gain of full and recovery_only to this effect. Modelled baselines. In rule_based and autoscale, covered faults are resolved after fixed latencies that we assumed (Section V). Their lead over the agents comes from those assumptions and is not a measurement of any deployed rule engine or autoscaler. Synthetic workload. Our data are a downscaled synthetic table with injected faults, not production traces, and real workloads may behave differently. One live model family. The live arm used one reasoning model and one faster model from a single provider at temperature zero, so the decision-quality findings (Section VI) are properties of that pairing and may not generalize. Unequal replicate counts. baseline, rule_based, autoscale, and full each have 80 runs, whereas the four single-agent ablations have half as many. We made this trade-off for cost reasons before the campaign started (Section V), and it leaves the ablations with wider confidence intervals than the headline comparisons. Nominal seed matching. Because the per-run seed depends on the configuration name (Section V), no two configurations share fault or human-latency randomness at a matched (scenario, replicate) index. We therefore used the unpaired rank-sum test as the primary test, and the retained signed-rank statistic should not be read as evidence from a paired design. Detection-clock asymmetry. Non-agent configurations detect at injection by construction, whereas agent configurations start the MTTR clock only when the monitoring agent flags the fault (Section III). The agents' recovery times therefore omit a detection interval that the non-agent figures include, which favors the agents in every MTTR comparison. Cost measurement. The metered term of Eq. (4) is integrated between two resource samples per run, which spans the 300 s control loop for agent configurations but only a few seconds for non-agent ones. In addition, the Airflow pool size and the streaming-worker target carry over from one run to the next because the campaign never reset them. Cost comparisons therefore rest mainly on the provisioning term, an assumption whose break-even point Section VI-G reports, and the metered part should not be read as a like-for-like measurement.

2) Scope of the Testbed: The testbed is a single machine. The resource-contention scenario applies real CPU stress, which is why runs cannot be parallelized and why the size of the campaign is bounded. The policy engine is pinned to one release (Table I), and lesson L6 (Section VII-F) shows that a newer release from the same vendor would have changed the adversarial result had the version not been pinned. We disclose this as a property of our evaluation, not as a defect we investigated in the policy engine. Only four fault types were injected, whereas a real pipeline's incident taxonomy is larger, and lesson L4 (Section VII-D) shows that mapping an injected fault type onto a real incident type is not automatic.

3) Evidence Gaps: Two components described in Section IV are implemented but not evaluated, and a third gap lies within a contribution that is otherwise evaluated. Cross-model generalization. The harness that could test the base paper's claim of model independence is unit-tested, but its live sweep against real provider models has not been run, so nothing here is evidence for or against that claim. The trust core. Graduated autonomy, the kill switch, and the blast-radius cap are implemented and unit-tested, but every configuration ran in one fixed mode, so no mode transition or kill-switch-to-halt latency was measured. Cost. Internally the cost model of Eq. (4) splits compute from storage, but the per-run data we committed store only the total, and the metered term carries the confound described above.

Each gap can be closed with a specific, limited piece of work. The cross-model sweep should run on the same scenario set against at least two live providers. A separate drill should engage the kill switch and a mode transition under load and time both. For cost, the harness should reset the pool size and worker target before every run, sample resources at a fixed rate over a window that is the same for all configurations, and keep the ledger's component breakdown. The first two gaps are missing evidence. The third is a flaw in how we measured, and a re-run with these changes would fix it.

4) What Would Change Our Conclusion: Suppose the live-versus-mock comparison of Section VI-E were repeated with another provider. If it showed the same order-of-magnitude gap, the central claim would be much stronger. If it did not, the gap would look like a feature of this particular model pairing and not of mock evaluation in general, and we would report that outcome just as plainly.

IX. CONCLUSION AND FUTURE WORK

We implemented and evaluated a policy-gated multi-agent design for cloud pipeline governance using a live remote language model, explicit non-agent baselines, an adversarial evaluation against an independent oracle, and closed-form sensitivity analyses. The design holds up only in part. Compared with static orchestration, it shortens recovery time, though by less than the original report claimed. Even that reduction comes mostly from the simulated engineer, not the agents. It lowers cost by more than claimed, but nearly all of that saving comes from an assumed provisioning gap. Manual interventions go up, and two idealized non-LLM comparators recover faster still. The result we consider most important concerns the evaluation, not the design. With the same code path and the same configuration, a deterministic mock and a live model lead to very different conclusions. Recovery times differ by about three orders of magnitude, and the mock reports a perfect correct-mitigation score that

the live model comes nowhere near. For systems like this one, knowing which evaluation path produced a number matters at least as much as the number. We keep the practical conclusion narrow on purpose. It says nothing about LLM agents in operations in general. It does say that a governance claim should come with an account of how it was evaluated, and that mock and live numbers are separate findings until both have been measured.

We see two directions for future work. One is to collect the evidence this campaign lacks: a live cross-model sweep to test model independence, a test of the trust core's mode transitions and kill-switch latency under load, a pre-registered re-run with the corrected simulator, per-run state resets and a fixed cost window, and deployed controllers and real production traces in place of the two modelled baselines (Section VIII). The other is design work: replacing the static agent-priority bid with an explicit negotiation protocol, enabling the bounded-adaptation mechanism that is currently disabled to preserve determinism, and extending the evaluation to multi-cloud deployments and to formal verification of the policy layer.

DATA AND CODE AVAILABILITY

The implementation, the Rego policies, the per-run campaign data for all 560 runs, the adversarial-corpus results, and the scripts that regenerate every table, figure, and reported number in this paper are available at https://github.com/bodapatisaikrishna/agentic-cloud-pipeline-governance.

ACKNOWLEDGMENT

Generative AI tools (Claude, Anthropic) were used to assist with software development, analysis scripts, and editing of the manuscript text. The authors reviewed all AI-assisted output, checked every reported result against the committed campaign data, and take full responsibility for the content of this paper.

References

[1] H. Foidl, V. Golendukhina, R. Ramler, and M. Felderer, "Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers," J. Syst. Softw., vol. 207, p. 111855, 2024, doi: 10.1016/j.jss.2023.111855.

[2] Y. Chen et al., "Automatic root cause analysis via large language models for cloud incidents," in Proc. 19th Eur. Conf. Comput. Syst. (EuroSys), 2024, pp. 674–688, doi: 10.1145/3627703.3629553.

[3] Z. Wang et al., "RCAgent: Cloud root cause analysis by autonomous agents with tool-augmented large language models," in Proc. 33rd ACM Int. Conf. Inf. Knowl. Manage. (CIKM), 2024, pp. 4966–4974, doi: 10.1145/3627673.3680016.

[4] Y. Luo et al., "OpsAgent: An evolving multi-agent system for incident management in microservices," arXiv:2510.24145, 2025, doi: 10.48550/arXiv.2510.24145.

[5] A. M. Kirubakaran et al., "Governing cloud data pipelines with agentic AI," arXiv:2512.23737, 2025, doi: 10.48550/arXiv.2512.23737.

[6] X. Liu, T. Chen, W. Li, X. Hu, and H. Wei, "The sim-to-real gap of foundation model agents: A unified MDP perspective," arXiv:2606.07017, 2026, KDD 2026 Blue Sky Ideas Track, doi: 10.48550/arXiv.2606.07017.

[7] M. Zhu et al., "RealUserSim: Bridging the reality gap in agent benchmarking via grounded user simulation," arXiv:2605.20204, 2026, doi: 10.48550/arXiv.2605.20204.

[8] A. Gupta, "ReliabilityBench: Evaluating LLM agent reliability under production-like stress conditions," arXiv:2601.06112, 2026, doi: 10.48550/arXiv.2601.06112.

[9] K. Rzadca et al., "Autopilot: Workload autoscaling at Google," in Proc. 15th Eur. Conf. Comput. Syst. (EuroSys), 2020, doi: 10.1145/3342195.3387524.

[10] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track, 2024, doi: 10.48550/arXiv.2406.13352.

[11] H. Zhang et al., "Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in Proc. 13th Int. Conf. Learn. Represent. (ICLR), 2025, doi: 10.48550/arXiv.2410.02644.

[12] A. Joshi, T. Finin, K. P. Joshi, and L. Kagal, "Deontic policies for runtime governance of agentic AI systems," arXiv:2606.19464, 2026, doi: 10.48550/arXiv.2606.19464.

[13] A. C. Sanna, "Cross-LLM generalization of behavioral backdoor detection in AI agent supply chains," arXiv:2511.19874, 2025, doi: 10.48550/arXiv.2511.19874.

[14] L. Zhang, Y.-Y. Chu, and R. Krishnan, "Toward agentic governance: What shapes LLM-agent intervention in public forums?" arXiv:2606.00603, 2026, doi: 10.48550/arXiv.2606.00603.

[15] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in Proc. 11th Int. Conf. Learn. Represent. (ICLR), 2023, doi: 10.48550/arXiv.2210.03629.

[16] F. F. Xu et al., "TheAgentCompany: Benchmarking LLM agents on consequential real world tasks," arXiv:2412.14161, 2024, doi: 10.48550/arXiv.2412.14161.

[17] J. Rystrøm, C. Schmitz, K. Korgul, J. Batzner, and C. Russell, "Agent benchmarks fail public sector requirements," arXiv:2601.20617, 2026, doi: 10.48550/arXiv.2601.20617.

[18] D. Hadfield-Menell, A. Dragan, P. Abbeel, and S. Russell, "The off-switch game," in Proc. 26th Int. Joint Conf. Artif. Intell. (IJCAI), 2017, pp. 220–227, doi: 10.24963/ijcai.2017/32.

[19] H. Zheng, Q. Dong, R. K. Depena, J. D. Bhatia, F. Xiao, and P. Xu, "Separating capability from permission: A governance framework for agentic AI autonomy levels," arXiv:2607.23438, 2026, doi: 10.48550/arXiv.2607.23438.

[20] Cloud Native Computing Foundation, "Open Policy Agent: policy-based control for cloud native environments," https://www.openpolicyagent.org, 2016, accessed 2026-09-21.

[21] K. Zhang, Y. Peng, K. Jiang, and Y. Zhang, "ActGov: Governing LLM agent actions via policy-constrained validation," arXiv:2609.24446, 2026, doi: 10.48550/arXiv.2609.24446.

[22] Y. Chen et al., "AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds," arXiv:2501.06706, 2025, doi: 10.48550/arXiv.2501.06706.

[23] J. Barnes, "OpenSec: Measuring incident response agent calibration under adversarial evidence," arXiv:2601.21083, 2026, doi: 10.48550/arXiv.2601.21083.

[24] A. Basiri et al., "Chaos engineering," IEEE Software, vol. 33, no. 3, pp. 35–41, 2016, doi: 10.1109/MS.2016.60.

[25] B. Efron, "Bootstrap methods: Another look at the jackknife," Ann. Statist., vol. 7, no. 1, 1979, doi: 10.1214/aos/1176344552.

[26] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," Ann. Math. Statist., vol. 18, no. 1, pp. 50–60, 1947, doi: 10.1214/aoms/1177730491.

[27] F. Wilcoxon, "Individual comparisons by ranking methods," Biometrics Bull., vol. 1, no. 6, p. 80, 1945, doi: 10.2307/3001968.

[28] S. Holm, "A simple sequentially rejective multiple test procedure," Scand. J. Statist., vol. 6, no. 2, pp. 65–70, 1979.

[29] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions," Psychol. Bull., vol. 114, no. 3, pp. 494–509, 1993, doi: 10.1037/0033-2909.114.3.494.

[30] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," J. Amer. Statist. Assoc., vol. 22, no. 158, pp. 209–212, 1927, doi: 10.1080/01621459.1927.10502953.